

Lecture Notes for Chapter 24: Single-Source Shortest Paths

Shortest paths

How to find the shortest route between two points on a map.

Input:

- Directed graph $G = (V, E)$
- Weight function $w : E \rightarrow \mathbb{R}$

Weight of path $p = \langle v_0, v_1, \dots, v_k \rangle$

$$= \sum_{i=1}^k w(v_{i-1}, v_i)$$

$$= \text{sum of edge weights on path } p.$$

Shortest-path weight u to v :

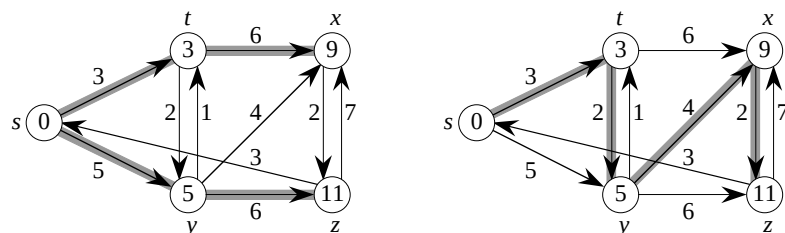
$$\delta(u, v) = \begin{cases} \min \{w(p) : u \xrightarrow{p} v\} & \text{if there exists a path } u \rightsquigarrow v, \\ \infty & \text{otherwise.} \end{cases}$$

Shortest path u to v is any path p such that $w(p) = \delta(u, v)$.

Example

shortest paths from s

[δ values appear inside vertices. Shaded edges show shortest paths.]



This example shows that the shortest path might not be unique.

It also shows that when we look at shortest paths from one vertex to all other vertices, the shortest paths are organized as a tree.

Can think of weights as representing any measure that

- accumulates linearly along a path, and
- we want to minimize.

Examples: time, cost, penalties, loss.

Generalization of breadth-first search to weighted graphs.

Variants

- **Single-source:** Find shortest paths from a given **source** vertex $s \in V$ to every vertex $v \in V$.
- **Single-destination:** Find shortest paths to a given destination vertex.
- **Single-pair:** Find shortest path from u to v . No way known that's better in worst case than solving single-source.
- **All-pairs:** Find shortest path from u to v for all $u, v \in V$. We'll see algorithms for all-pairs in the next chapter.

Negative-weight edges

OK, as long as no negative-weight cycles are reachable from the source.

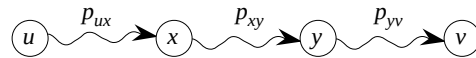
- If we have a negative-weight cycle, we can just keep going around it, and get $w(s, v) = -\infty$ for all v on the cycle.
- But OK if the negative-weight cycle is not reachable from the source.
- Some algorithms work only if there are no negative-weight edges in the graph. We'll be clear when they're allowed and not allowed.

Optimal substructure

Lemma

Any subpath of a shortest path is a shortest path.

Proof Cut-and-paste.



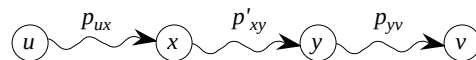
Suppose this path p is a shortest path from u to v .

Then $\delta(u, v) = w(p) = w(p_{ux}) + w(p_{xy}) + w(p_{yv})$.

Now suppose there exists a shorter path $x \overset{p'_{xy}}{\rightsquigarrow} y$.

Then $w(p'_{xy}) < w(p_{xy})$.

Construct p' :



Then

$$\begin{aligned} w(p') &= w(p_{ux}) + w(p'_{xy}) + w(p_{yv}) \\ &< w(p_{ux}) + w(p_{xy}) + w(p_{yv}) \\ &= w(p). \end{aligned}$$

Contradicts the assumption that p is a shortest path.

■ (lemma)

Cycles

Shortest paths can't contain cycles:

- Already ruled out negative-weight cycles.
- Positive-weight $\Rightarrow$ we can get a shorter path by omitting the cycle.
- Zero-weight: no reason to use them $\Rightarrow$ assume that our solutions won't use them.

Output of single-source shortest-path algorithm

For each vertex $v \in V$:

- $v.d = \delta(s, v)$.
 - Initially, $v.d = \infty$.
 - Reduces as algorithms progress. But always maintain $v.d \geq \delta(s, v)$.
 - Call $v.d$ a **shortest-path estimate**.
- $v.\pi$ = predecessor of v on a shortest path from s .
 - If no predecessor, $v.\pi = \text{NIL}$.
 - π induces a tree—**shortest-path tree**.
 - We won't prove properties of π in lecture—see text.

Initialization

All the shortest-paths algorithms start with INIT-SINGLE-SOURCE.

INIT-SINGLE-SOURCE(G, s)

```

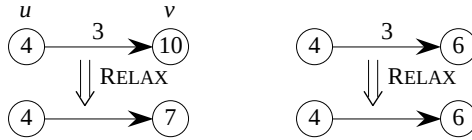
for each  $v \in G.V$ 
     $v.d = \infty$ 
     $v.\pi = \text{NIL}$ 
 $s.d = 0$ 
```

Relaxing an edge (u, v)

Can we improve the shortest-path estimate for v by going through u and taking (u, v) ?

$\text{RELAX}(u, v, w)$

if $v.d > u.d + w(u, v)$
 $v.d = u.d + w(u, v)$
 $v.\pi = u$



For all the single-source shortest-paths algorithms we'll look at,

- start by calling INIT-SINGLE-SOURCE,
- then relax edges.

The algorithms differ in the order and how many times they relax each edge.

Shortest-paths properties

Based on calling INIT-SINGLE-SOURCE once and then calling RELAX zero or more times.

Triangle inequality

For all $(u, v) \in E$, we have $\delta(s, v) \leq \delta(s, u) + w(u, v)$.

Proof Weight of shortest path $s \rightsquigarrow v$ is $\leq$ weight of any path $s \rightsquigarrow v$. Path $s \rightsquigarrow u \rightarrow v$ is a path $s \rightsquigarrow v$, and if we use a shortest path $s \rightsquigarrow u$, its weight is $\delta(s, u) + w(u, v)$. ■

Upper-bound property

Always have $v.d \geq \delta(s, v)$ for all v . Once $v.d = \delta(s, v)$, it never changes.

Proof Initially true.

Suppose there exists a vertex such that $v.d < \delta(s, v)$.

Without loss of generality, v is first vertex for which this happens.

Let u be the vertex that causes $v.d$ to change.

Then $v.d = u.d + w(u, v)$.

So,

$$\begin{aligned}
 v.d &< \delta(s, v) \\
 &\leq \delta(s, u) + w(u, v) \quad (\text{triangle inequality}) \\
 &\leq u.d + w(u, v) \quad (v \text{ is first violation}) \\
 \Rightarrow v.d &< u.d + w(u, v) .
 \end{aligned}$$

Contradicts $v.d = u.d + w(u, v)$.

Once $v.d$ reaches $\delta(s, v)$, it never goes lower. It never goes up, since relaxations only lower shortest-path estimates. ■

No-path property

If $\delta(s, v) = \infty$, then $v.d = \infty$ always.

Proof $v.d \geq \delta(s, v) = \infty \Rightarrow v.d = \infty$. ■

Convergence property

If $s \rightsquigarrow u \rightarrow v$ is a shortest path, $u.d = \delta(s, u)$, and we call RELAX(u, v, w), then $v.d = \delta(s, v)$ afterward.

Proof After relaxation:

$$\begin{aligned} v.d &\leq u.d + w(u, v) && \text{(RELAX code)} \\ &= \delta(s, u) + w(u, v) \\ &= \delta(s, v) && \text{(lemma—optimal substructure)} \end{aligned}$$

Since $v.d \geq \delta(s, v)$, must have $v.d = \delta(s, v)$. ■

Path relaxation property

Let $p = \langle v_0, v_1, \dots, v_k \rangle$ be a shortest path from $s = v_0$ to v_k . If we relax, *in order*, $(v_0, v_1), (v_1, v_2), \dots, (v_{k-1}, v_k)$, even intermixed with other relaxations, then $v_k.d = \delta(s, v_k)$.

Proof Induction to show that $v_i.d = \delta(s, v_i)$ after (v_{i-1}, v_i) is relaxed.

Basis: $i = 0$. Initially, $v_0.d = 0 = \delta(s, v_0) = \delta(s, s)$.

Inductive step: Assume $v_{i-1}.d = \delta(s, v_{i-1})$. Relax (v_{i-1}, v_i) . By convergence property, $v_i.d = \delta(s, v_i)$ afterward and $v_i.d$ never changes. ■

The Bellman-Ford algorithm

- Allows negative-weight edges.
- Computes $v.d$ and $v.\pi$ for all $v \in V$.
- Returns TRUE if no negative-weight cycles reachable from s , FALSE otherwise.

```

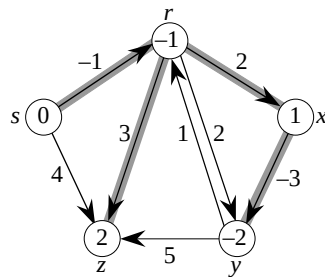
BELLMAN-FORD( $G, w, s$ )
  INIT-SINGLE-SOURCE( $G, s$ )
  for  $i = 1$  to  $|G.V| - 1$ 
    for each edge  $(u, v) \in G.E$ 
      RELAX( $u, v, w$ )
  for each edge  $(u, v) \in G.E$ 
    if  $v.d > u.d + w(u, v)$ 
      return FALSE
  return TRUE

```

Core: The nested **for** loops relax all edges $|V| - 1$ times.

Time: $\Theta(VE)$.

Example



Values you get on each pass and how quickly it converges depends on order of relaxation.

But guaranteed to converge after $|V| - 1$ passes, assuming no negative-weight cycles.

Proof Use path-relaxation property.

Let v be reachable from s , and let $p = \langle v_0, v_1, \dots, v_k \rangle$ be a shortest path from s to v , where $v_0 = s$ and $v_k = v$. Since p is acyclic, it has $\leq |V| - 1$ edges, so $k \leq |V| - 1$.

Each iteration of the **for** loop relaxes all edges:

- First iteration relaxes (v_0, v_1) .
- Second iteration relaxes (v_1, v_2) .
- k th iteration relaxes (v_{k-1}, v_k) .

By the path-relaxation property, $v.d = v_k.d = \delta(s, v_k) = \delta(s, v)$. ■

How about the TRUE/FALSE return value?

- Suppose there is no negative-weight cycle reachable from s .

At termination, for all $(u, v) \in E$,

$$\begin{aligned}
 v.d &= \delta(s, v) \\
 &\leq \delta(s, u) + w(u, v) \quad (\text{triangle inequality}) \\
 &= u.d + w(u, v).
 \end{aligned}$$

So BELLMAN-FORD returns TRUE.

- Now suppose there exists negative-weight cycle $c = \langle v_0, v_1, \dots, v_k \rangle$, where $v_0 = v_k$, reachable from s .

$$\text{Then } \sum_{i=1}^k (v_{i-1}, v_i) < 0.$$

Suppose (for contradiction) that BELLMAN-FORD returns TRUE.

Then $v_i.d \leq v_{i-1}.d + w(v_{i-1}, v_i)$ for $i = 1, 2, \dots, k$.

Sum around c :

$$\begin{aligned} \sum_{i=1}^k v_i.d &\leq \sum_{i=1}^k (v_{i-1}.d + w(v_{i-1}, v_i)) \\ &= \sum_{i=1}^k v_{i-1}.d + \sum_{i=1}^k w(v_{i-1}, v_i) \end{aligned}$$

Each vertex appears once in each summation $\sum_{i=1}^k v_i.d$ and $\sum_{i=1}^k v_{i-1}.d \Rightarrow$

$$0 \leq \sum_{i=1}^k w(v_{i-1}, v_i).$$

Contradicts c being a negative-weight cycle. ■

Single-source shortest paths in a directed acyclic graph

Since a dag, we're guaranteed no negative-weight cycles.

DAG-SHORTEST-PATHS(G, w, s)

topologically sort the vertices

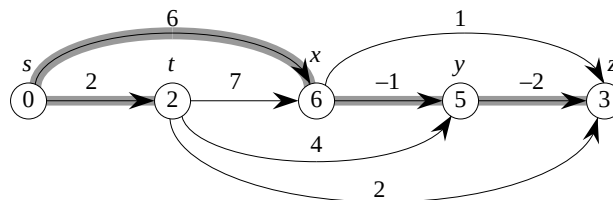
INIT-SINGLE-SOURCE(G, s)

for each vertex u , taken in topologically sorted order

for each vertex $v \in G.Adj[u]$

 RELAX(u, v, w)

Example



Time

$\Theta(V + E)$.

Correctness

Because we process vertices in topologically sorted order, edges of *any* path must be relaxed in order of appearance in the path.

⇒ Edges on any shortest path are relaxed in order.

⇒ By path-relaxation property, correct. ■

Dijkstra's algorithm

No negative-weight *edges*.

Essentially a weighted version of breadth-first search.

- Instead of a FIFO queue, uses a priority queue.
- Keys are shortest-path weights ($v.d$).

Have two sets of vertices:

- S = vertices whose final shortest-path weights are determined,
- Q = priority queue = $V - S$.

DIJKSTRA(G, w, s)

INIT-SINGLE-SOURCE(G, s)

$S = \emptyset$

$Q = G.V$ // i.e., insert all vertices into Q

while $Q \neq \emptyset$

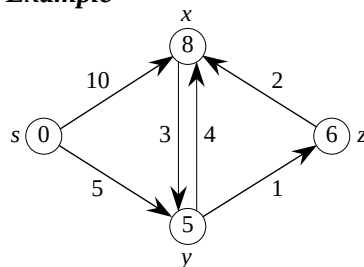
$u = \text{EXTRACT-MIN}(Q)$

$S = S \cup \{u\}$

for each vertex $v \in G.Adj[u]$

 RELAX(u, v, w)

- Looks a lot like Prim's algorithm, but computing $v.d$, and using shortest-path weights as keys.
- Dijkstra's algorithm can be viewed as greedy, since it always chooses the "lightest" ("closest"?) vertex in $V - S$ to add to S .

Example

Order of adding to S : s, y, z, x .

Correctness

Loop invariant: At the start of each iteration of the **while** loop, $v.d = \delta(s, v)$ for all $v \in S$.

Initialization: Initially, $S = \emptyset$, so trivially true.

Termination: At end, $Q = \emptyset \Rightarrow S = V \Rightarrow v.d = \delta(s, v)$ for all $v \in V$.

Maintenance: Need to show that $u.d = \delta(s, u)$ when u is added to S in each iteration.

Suppose there exists u such that $u.d \neq \delta(s, u)$. Without loss of generality, let u be the first vertex for which $u.d \neq \delta(s, u)$ when u is added to S .

Observations:

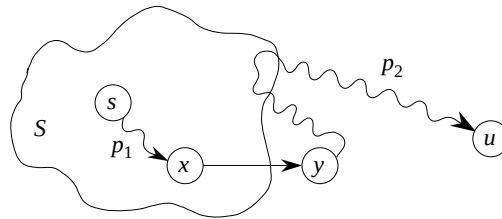
- $u \neq s$, since $s.d = \delta(s, s) = 0$.
- Therefore, $s \in S$, so $S \neq \emptyset$.
- There must be some path $s \rightsquigarrow u$, since otherwise $u.d = \delta(s, u) = \infty$ by no-path property.

So, there's a path $s \rightsquigarrow u$.

Then there's a shortest path $s \overset{p}{\rightsquigarrow} u$.

Just before u is added to S , path p connects a vertex in S (i.e., s) to a vertex in $V - S$ (i.e., u).

Let y be first vertex along p that's in $V - S$, and let $x \in S$ be y 's predecessor.



Decompose p into $s \overset{p_1}{\rightsquigarrow} x \rightarrow y \overset{p_2}{\rightsquigarrow} u$. (Could have $x = s$ or $y = u$, so that p_1 or p_2 may have no edges.)

Claim

$y.d = \delta(s, y)$ when u is added to S .

Proof $x \in S$ and u is the first vertex such that $u.d \neq \delta(s, u)$ when u is added to $S \Rightarrow x.d = \delta(s, x)$ when x is added to S . Relaxed (x, y) at that time, so by the convergence property, $y.d = \delta(s, y)$. ■ (claim)

Now can get a contradiction to $u.d \neq \delta(s, u)$:

y is on shortest path $s \rightsquigarrow u$, and all edge weights are nonnegative

$$\Rightarrow \delta(s, y) \leq \delta(s, u) \Rightarrow$$

$$y.d = \delta(s, y)$$

$$\leq \delta(s, u)$$

$$\leq u.d \quad (\text{upper-bound property}) .$$

Also, both y and u were in Q when we chose u , so

$$u.d \leq y.d \Rightarrow u.d = y.d.$$

Therefore, $y.d = \delta(s, y) = \delta(s, u) = u.d$.

Contradicts assumption that $u.d \neq \delta(s, u)$. Hence, Dijkstra's algorithm is correct. ■

Analysis

Like Prim's algorithm, depends on implementation of priority queue.

- If binary heap, each operation takes $O(\lg V)$ time $\Rightarrow O(E \lg V)$.
- If a Fibonacci heap:
 - Each EXTRACT-MIN takes $O(1)$ amortized time.
 - There are $O(V)$ other operations, taking $O(\lg V)$ amortized time each.
 - Therefore, time is $O(V \lg V + E)$.

Difference constraints

Given a set of inequalities of the form $x_j - x_i \leq b_k$.

- x 's are variables, $1 \leq i, j \leq n$,
- b 's are constants, $1 \leq k \leq m$.

Want to find a set of values for the x 's that satisfy all m inequalities, or determine that no such values exist. Call such a set of values a **feasible solution**.

Example

$$\begin{aligned} x_1 - x_2 &\leq 5 \\ x_1 - x_3 &\leq 6 \\ x_2 - x_4 &\leq -1 \\ x_3 - x_4 &\leq -2 \\ x_4 - x_1 &\leq -3 \end{aligned}$$

Solution: $x = (0, -4, -5, -3)$

Also: $x = (5, 1, 0, 2) = [\text{above solution}] + 5$

Lemma

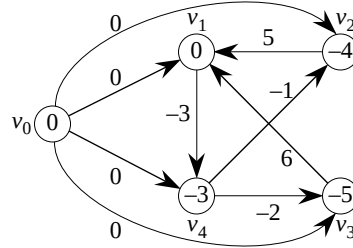
If x is a feasible solution, then so is $x + d$ for any constant d .

Proof x is a feasible solution $\Rightarrow x_j - x_i \leq b_k$ for all i, j, k
 $\Rightarrow (x_j + d) - (x_i + d) \leq b_k$. ■ (lemma)

Constraint graph

$G = (V, E)$, weighted, directed.

- $V = \{v_0, v_1, v_2, \dots, v_n\}$: one vertex per variable + v_0
- $E = \{(v_i, v_j) : x_j - x_i \leq b_k \text{ is a constraint}\} \cup \{(v_0, v_1), (v_0, v_2), \dots, (v_0, v_n)\}$
- $w(v_0, v_j) = 0$ for all j
- $w(v_i, v_j) = b_k$ if $x_j - x_i \leq b_k$



Theorem

Given a system of difference constraints, let $G = (V, E)$ be the corresponding constraint graph.

1. If G has no negative-weight cycles, then

$$x = (\delta(v_0, v_1), \delta(v_0, v_2), \dots, \delta(v_0, v_n))$$

is a feasible solution.

2. If G has a negative-weight cycle, then there is no feasible solution.

Proof

1. Show no negative-weight cycles $\Rightarrow$ feasible solution.

Need to show that $x_j - x_i \leq b_k$ for all constraints. Use

$$x_j = \delta(v_0, v_j)$$

$$x_i = \delta(v_0, v_i)$$

$$b_k = w(v_i, v_j) .$$

By the triangle inequality,

$$\delta(v_0, v_j) \leq \delta(v_0, v_i) + w(v_i, v_j)$$

$$x_j \leq x_i + b_k$$

$$x_j - x_i \leq b_k .$$

Therefore, feasible.

2. Show negative-weight cycles $\Rightarrow$ no feasible solution.

Without loss of generality, let a negative-weight cycle be $c = \langle v_1, v_2, \dots, v_k \rangle$, where $v_1 = v_k$. (v_0 can't be on c , since v_0 has no entering edges.) c corresponds to the constraints

$$x_2 - x_1 \leq w(v_1, v_2) ,$$

$$x_3 - x_2 \leq w(v_2, v_3) ,$$

$$\vdots$$

$$x_{k-1} - x_{k-2} \leq w(v_{k-2}, v_{k-1}) ,$$

$$x_k - x_{k-1} \leq w(v_{k-1}, v_k) .$$

If x is a solution satisfying these inequalities, it must satisfy their sum.

So add them up.

Each x_i is added once and subtracted once. ($v_1 = v_k \Rightarrow x_1 = x_k$.)

We get $0 \leq w(c)$.

But $w(c) < 0$, since c is a negative-weight cycle.

Contradiction $\Rightarrow$ no such feasible solution x exists.

■ (theorem)

How to find a feasible solution

1. Form constraint graph.

- $n + 1$ vertices.
- $m + n$ edges.
- $\Theta(m + n)$ time.

2. Run BELLMAN-FORD from v_0 .

- $O((n + 1)(m + n)) = O(n^2 + nm)$ time.

3. If BELLMAN-FORD returns FALSE $\Rightarrow$ no feasible solution.

If BELLMAN-FORD returns TRUE $\Rightarrow$ set $x_i = \delta(v_0, v_i)$ for all i .